



CWE-25: Path Traversal: '/../filedir'

Weakness ID: 25

Vulnerability Mapping: ALLOWED**Abstraction:** Variant

View customized information:

Conceptual

Operational

Mapping
Friendly**Complete**

Custom

▼ Description

The product uses external input to construct a pathname that should be within a restricted directory, but it does not properly neutralize "../" sequences that can resolve to a location that is outside of that directory.

▼ Extended Description

This allows attackers to traverse the file system to access files or directories that are outside of the restricted directory.

Sometimes a program checks for "../" at the beginning of the input, so a "../" can bypass that check.

▼ Common Consequences



Impact

*Read Files or
Directories; Modify
Files or Directories*

Details

Scope: Confidentiality, Integrity

▼ Potential Mitigations

Phase(s)	Mitigation
Implementation	Strategy: Input Validation Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When validating filenames, use stringent allowlists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses such as CWE-23, and exclude directory separators such as "/" to avoid CWE-36. Use a list of allowable file extensions, which will help to avoid CWE-434. Do not rely exclusively on a filtering mechanism that removes potentially dangerous characters. This is equivalent to a denylist, which may be incomplete (CWE-184). For example, filtering "/" is insufficient protection if the filesystem also supports the use of "\" as a directory separator. Another possible error could occur when the filtering is applied in a way that still produces dangerous data (CWE-182). For example, if "../" sequences are removed from the ".../.../" string in a sequential fashion, two instances of "../" would be removed from the original string, but the remaining characters would still form the "../" string.
Implementation	Strategy: Input Validation Inputs should be decoded and canonicalized to the application's current internal representation before being validated (CWE-180). Make sure that the application does

not decode the same input twice ([CWE-174](#)). Such errors could be used to bypass allowlist validation schemes by introducing dangerous inputs after they have been checked.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf		23	Relative Path Traversal

Modes Of Introduction



Phase	Note
Implementation	

Applicable Platforms



Languages	Class: Not Language-Specific (<i>Undetermined Prevalence</i>)
-----------	---

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2022-20775	A cloud management tool allows attackers to bypass the restricted shell using path traversal sequences like "../" in the USER environment variable.

Weakness Ordinalities

Ordinality	Description
Primary	(where the weakness exists independent of other weaknesses)

Detection Methods

Method	Details
Automated Static Analysis	Automated static analysis, commonly referred to as Static Application Security Testing (SAST), can find some instances of this weakness by analyzing source code (or binary/compiled code) without having to execute it. Typically, this is done by building a model of data flow and control flow, then searching for potentially-vulnerable patterns that connect "sources" (origins of input) with "sinks" (destinations where the data interacts with external components, a lower layer such as the OS, etc.)

Functional Areas

- File Processing

Affected Resources

- File or Directory

Memberships



Nature	Type	ID	Name
MemberOf		981	SFP Secondary Cluster: Path Traversal
MemberOf		1404	Comprehensive Categorization: File Handling

Vulnerability Mapping Notes

Usage	ALLOWED (this CWE ID may be used to map to real-world vulnerabilities)
-------	---

Reason	Acceptable-Use
Rationale	This CWE entry is at the Variant level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

▼ Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name
PLOVER			'../../../../filedir
Software Fault Patterns	SFP16		Path Traversal

▼ Content History

▼ Submissions		
Submission Date	Submitter	Organization
2006-07-19 <i>(CWE Draft 3, 2006-07-19)</i>	PLOVER	
► Modifications		
► Previous Entry Names		